.things that Matter

CTO Next Steps & Action Plan

Document Information	
---	---
Company Name	**.things that Matter**
Document Type	**CTO Action Plan**
Version	**1.0**
Date	**January 21, 2026**
Author	**Matome Mboweni, CTO**

Executive Summary

Current Status: Sprint 0 (Setup Phase) - Week 1, Day 1 **Next Milestone:** Phase 0 Go/No-Go Decision (Feb 28, 2026) **Timeline:** 11 months to public launch (Dec 15, 2026)

This document outlines immediate next steps, priorities, and action items for the CTO to execute over the next 2-4 weeks.

Immediate Priorities (This Week: Jan 21-27, 2026)

🎯 Priority 1: Complete Sprint 0 Setup Tasks

Task	Status	Deadline	Owner	Notes
Sign new company contract	⏳ Pending	Jan 31, 2026	CEO + CTO	Legal foundation
Set up development environment	⏳ In Progress	Feb 2, 2026	CTO	Cursor Pro, VS Code, Rust
Create GitHub organization	⏳ Pending	Jan 25, 2026	CTO	`thingsthatmatter` org
Initialize monorepo structure	⏳ Pending	Jan 26, 2026	CTO	`packages/` directory
Set up Supabase project	⏳ Pending	Jan 28, 2026	CTO	Database schema
Configure Vercel deployment	⏳ Pending	Jan 29, 2026	CTO	CI/CD pipeline

Set up Cloudflare R2 bucket	**⏳ Pending**	**Jan 30, 2026**	**CTO**	**Artifact storage**

Begin Rust learning	**⏳ In Progress**	**Ongoing**	**CTO**	**Chapters 1-5**

🔧 Priority 2: Infrastructure Accounts Setup

Action Items:

1. **Create accounts** (if not already done): - ✅ GitHub Organization: `thingsthatmatter` - ⏳ Supabase account (free tier) - ⏳ Vercel account (hobby tier) - ⏳ Cloudflare account (R2 storage) - ⏳ Replicate account (CLIP API) - ⏳ Redis Cloud (free tier)

2. **Set up billing alerts** for all paid services: - Cloudflare R2: ~R630/month limit - Replicate CLIP: ~R540/month limit

3. **Create service accounts and API keys:** - Store securely in environment variables - Document in secure password manager

---

Week 2 Priorities (Jan 28 - Feb 2, 2026)

📋 Sprint 0 Completion Checklist

- [ ] All infrastructure accounts created and verified - [ ] Development environment fully operational - [ ] GitHub repo structure established - [ ] Database schema designed (initial draft) - [ ] CI/CD pipeline configured - [ ] Rust toolchain installed (`rustup` , `cargo`) - [ ] Rust Book: Completed Chapters 1-5

🚀 Phase 0 Preparation

Begin planning for Phase 0: Validation (Feb 3 - Mar 2)

1. **Steganography Research:** - Research XMP metadata injection libraries (Rust crates) - Research EOF binary append techniques - Identify test platforms (Discord, Telegram, Twitter, etc.) - Create Phase 0 testing plan

2. **Tool Preparation:** - Set up Rust project for steganography tools - Identify crates: `image` (image processing), `exif` (XMP), etc. - Create test image samples

---

Phase 0: Critical Validation (Feb 3 - Mar 2, 2026)

Week 3-4: Build Steganography Tools

Deliverables: - [ ] `xmp_inject.py` - XMP metadata injection tool - [ ] `eof_append.py` - EOF binary append tool - [ ] `extract_payload.py` - Payload extraction tool - [ ] Initial platform tests (5 platforms)

Technical Approach:

```
rust
// Example structure
xmp_inject.rs:
  - Read image (JPEG/PNG) using image crate
  - Inject XMP metadata with hash + gist
  - Save as polyglot image

eof_append.rs:
  - Read image
  - Append .thing binary to EOF
  - Validate file still displays correctly

extract_payload.rs:
  - Extract XMP beacon from image header
  - Extract EOF binary payload
  - Validate both are intact
```

Week 5: End-to-End Prototype

Build Proof of Concept:

```
Product JSON → Compiler → .thing File → Polyglot Image → Platform Upload → Extra
```

Components to Build: 1. Basic product JSON validator (using `serde_json` + `validator`) 2. Simple FlatBuffers serializer (Rust) 3. Ed25519 signing (using `ed25519-dalek` crate) 4. CLIP API integration test (using `reqwest`) 5. Image steganography pipeline (using `image` crate) 6. Extraction and verification

Success Criteria: - End-to-end flow works locally - Can extract and verify payload after platform upload - Document any issues or limitations

Week 6: Platform Testing & Go/No-Go Decision

Test Matrix (20 platforms):

Platform	Test Method	Expected Result	Actual Result	Survival?
---	---	---	---	---
Discord	**Upload/download**	**XMP + EOF intact**	**TBD**	❓
Telegram	**Send/receive**	**XMP + EOF intact**	**TBD**	❓
Twitter/X	**Tweet with image**	**XMP may strip**	**TBD**	❓
Instagram	**Post image**	**Likely strip**	**TBD**	❓
…	…	…	…	…

Decision Logic:

Survival Rate	Decision	Action Plan
≥70%	✅ **GO**	**Full steganography approach**
50-69%	⚠️ **HYBRID**	**Steg + matter:// URIs**
<50%	🔄 **PIVOT**	**matter:// as primary, steg secondary**

Phase 0 Report Deliverable: - Testing results for all 20 platforms - Survival rate calculation - Cost analysis (CLIP API) - Go/No-Go recommendation - Updated roadmap (if pivot needed)

Technical Learning Path

Rust Learning Schedule

Week 1-2 (Jan 21 - Feb 2): - [ ] Complete Rust Book Chapters 1-5 - [ ] Set up Rust development environment - [ ] Build simple CLI tool (hello world → file I/O)

Week 3-4 (Feb 3-16): - [ ] Rust Book Chapters 6-10 (Ownership, Structs, Enums) - [ ] Practice: Build simple FlatBuffers serializer - [ ] Study `ed25519-dalek` crate examples

Week 5-8 (Feb 17 - Mar 16): - [ ] Rust Book Chapters 11-15 (Testing, CLI, Collections) - [ ] Study `wasm-pack` for WASM compilation - [ ] Review existing Rust commerce/blockchain projects

Ongoing: - Practice Rust daily (even if just 30 minutes) - Join Rust Discord/forums for questions - Read Rust blogs and watch tutorials

Rust Implementation Focus (Sprint 2-3)

Priority: Build Rust compiler from the start, leveraging C/C++ background

Rust Stack: - `serde_json` + `validator` for validation - `flatbuffers` (Rust crate) - `ed25519-dalek` for Ed25519 signing - `image` crate for image manipulation - `exif` or `rexiv2` for XMP metadata - `zstd` for compression - `reqwest` for CLIP API calls - `tokio` for async operations

---

Infrastructure Setup Details

1. Supabase Project Setup

Steps: 1. Create new Supabase project 2. Design initial schema:

```
sql
   -- Merchants table
   -- Products table
   -- ThingFiles table (hash → metadata)
   -- Resolution logs
```

3. Set up Row Level Security (RLS) 4. Create API keys 5. Test connection from local dev

Schema Design: See Technical Architecture doc (Section 2.6)

2. Vercel Project Setup

Steps: 1. Create Vercel account (connect GitHub) 2. Create new project 3. Set up environment variables: - `SUPABASE_URL` - `SUPABASE_ANON_KEY` - `SUPABASE_SERVICE_KEY` - `CLOUDFLARE_R2_*` - `REPLICATE_API_KEY` - `SIGNING_PRIVATE_KEY` (Ed25519) 4. Configure Edge Functions 5. Test deployment

3. Cloudflare R2 Setup

Steps: 1. Create Cloudflare account 2. Enable R2 (pay-as-you-go) 3. Create bucket: `matter-artifacts` 4. Set up CORS for web access 5. Create public CDN endpoint 6. Test upload/download

Pricing: ~R630/month for 10K products (estimate)

4. Replicate API Setup

Steps: 1. Create Replicate account 2. Get API key 3. Test CLIP model:

```
rust
   use reqwest;
   // Example API call structure
   let response = reqwest::Client::new()
       .post("https://api.replicate.com/v1/predictions")
       .json(&json!({
           "version": "clip-vit-base-patch32",
           "input": {"image": image_url}
       }))
       .send()
       .await?;
```

4. Document cost per inference (~R0.40/image)

Risk Mitigation Actions

Immediate Risk Mitigations:

1. **Cursor Pro Limits:** - ✅ Document created (you're reading it) - ⏳ Batch work requests when possible - ⏳ Use slower requests for non-critical tasks - ⏳ Plan ahead to minimize back-and-forth

2. **Rust Learning Curve:** - ⏳ Start early (ongoing in Sprint 0) - ⏳ C/C++ background provides strong foundation - ⏳ Learn Rust as you build (practical approach) - ⏳ Allocate 3 hours/week minimum for learning

3. **Budget Management:** - ⏳ Set up billing alerts - ⏳ Monitor monthly spend closely - ⏳ Use free tiers wherever possible - ⏳ Keep 27% operating reserve untouched

4. **Steganography Validation:** - ⏳ Test early and often - ⏳ Have matter:// URI fallback ready - ⏳ Document all platform behaviors

---

Weekly Routine (Recommended)

Monday Morning (2 hours)

- Review previous week's progress - Plan week ahead - Update CEO on status - Prioritize tasks

Tuesday-Thursday (20 hours)

- Core development work - Infrastructure setup - Learning (Rust, new tech)

Friday (3 hours)

- Code review & cleanup - Documentation updates - Progress tracking - Next week prep

Weekend (Optional)

- Light reading/research - Experimentation - Rest (important!)

Total: ~30 hours/week (matches roadmap assumptions)

Communication Cadence

With CEO:

- **Weekly sync:** Every Monday (30 min) - **Monthly demo:** First Monday of month - **Blockers:** Immediate notification

Documentation:

- **Update docs:** Weekly (Friday) - **Progress tracking:** Update roadmap status - **Decision logs:** Document major technical decisions

---

Success Metrics (Sprint 0)

Metric	Target	Current	Status
--------	--------	---------	--------
Infrastructure accounts	**6/6**	**?**	⏳
Development environment	**100%**	**?**	⏳
GitHub org created	✅	**?**	⏳
Rust Chapters completed	**5/5**	**?**	⏳
Monorepo structure	✅	**?**	⏳
Supabase project	✅	**?**	⏳
Vercel deployment	✅	**?**	⏳
R2 bucket	✅	**?**	⏳

Immediate Action Items (Today/Tomorrow)

Today (Jan 21, 2026):

- [x] Review this document - [ ] Confirm contract signing timeline with CEO - [ ] Create GitHub organization - [ ] Set up Supabase account - [ ] Install Rust toolchain (`rustup install stable`) - [ ] Start Rust Book Chapter 1

Tomorrow (Jan 22, 2026):

- [ ] Set up Vercel account - [ ] Create Cloudflare R2 bucket - [ ] Design database schema (initial draft) - [ ] Initialize monorepo structure - [ ] Complete Rust Book Chapter 1-2 - [ ] Document any blockers

---

Decision Points Coming Up

Week 2 (Jan 28 - Feb 2):

- **Finalize database schema** (needed before Phase 0) - **Choose Rust crates** for implementation (research dependencies) - **Define API endpoints** (Shadow Index API design)

Phase 0 Week 4 (Feb 24-28):

- **Go/No-Go decision** on steganography approach - **Pivot decision** if survival rate <50% - **Updated roadmap** if approach changes

---

Resources & References

Documentation:

- `01-PROJECT-CHARTER.md` - Vision and objectives - `02-TECHNICAL-ARCHITECTURE.md` - System design - `03-PRODUCT-REQUIREMENTS-DOCUMENT.md` - Requirements - `04-DEVELOPMENT-ROADMAP.md` - Detailed timeline - `05-RISK-ASSESSMENT.md` - Risk mitigation - `07-SYSTEM-DIAGRAMS.md` - UML diagrams

External Resources:

- The Rust Book - FlatBuffers Guide - Replicate CLIP API - Ed25519 Cryptography

---

Notes & Open Questions

Technical Questions to Resolve:

- [ ] Best Rust crate for XMP injection? (`exif` , `rexiv2` , or custom?) - [ ] FlatBuffers Rust crate performance acceptable? - [ ] WASM size constraints for browser extension? - [ ] Best steganography technique for JPEG vs PNG? - [ ] Async runtime choice (`tokio` vs `async-std`)?

Business Questions to Clarify with CEO:

- [ ] Contract signing timeline? - [ ] Any changes to budget allocation? - [ ] Priority adjustments based on market feedback? - [ ] Partnership opportunities to explore?

Appendix: Quick Reference

Command Cheat Sheet:

```
bash
```

Rust

```
rustup install stable
cargo new matter-core --lib
cargo test
```

Git

```
git init
git remote add origin https://github.com/thingsthatmatter/...
git branch -M main
```

Rust

```
cargo new matter-core --lib
cargo add serde serde_json
cargo add flatbuffers
cargo add ed25519-dalek
cargo add image
cargo add reqwest --features json
cargo add tokio --features full
```

Environment Variables Template:

```bash
# .env file (never commit!)

SUPABASE_URL=https://xxx.supabase.co
SUPABASE_ANON_KEY=xxx
SUPABASE_SERVICE_KEY=xxx
CLOUDFLARE_R2_ACCOUNT_ID=xxx
CLOUDFLARE_R2_ACCESS_KEY_ID=xxx
CLOUDFLARE_R2_SECRET_ACCESS_KEY=xxx
REPLICATE_API_KEY=xxx
SIGNING_PRIVATE_KEY=xxx  # Ed25519 private key (base64)
```

Document Version: 1.0 **Last Updated:** January 21, 2026 **Next Review:** January 28, 2026 (End of Sprint 0, Week 1)

Remember: You're building something revolutionary. Take it one step at a time, validate early, and don't be afraid to pivot if needed. The Phase 0 validation is critical—it will determine the entire technical approach.

Good luck! 🚀